more detail in Chapter 7. Teams that fail to spend a reasonable amount of time on this task will find that their project contains expensive rework, cost overruns, poor product quality, late delivery times, dissatisfied customers, and poor team morale. Requirements engineering cannot be neglected, nor can it be allowed to iterate endlessly before proceeding to product construction.

It's reasonable to ask what best practices should be followed to achieve thorough and agile requirements engineering. Scott Ambler [Amb12] suggests several best practices for agile requirements definition:

1. Encourage active stakeholder participation by matching their availability and valuing their input.
2. Use simple models (e.g., Post-it notes, fast sketches, user stories) to reduce barriers to participation.
3. Take time to explain your requirement representation techniques before using them.
4. Adopt stakeholder terminology, and avoid technical jargon whenever possible.
5. Use a breadth-first approach to get the big picture of the project done before getting bogged down in details.
6. Allow the development team to refine (with stakeholder input) requirement details "just in time" as user stories are scheduled to be implemented.
7. Treat the list of features to be implemented like a prioritized list, and implement the most important user stories first.
8. Collaborate closely with your stakeholders and only document requirements at a level that is useful to all when creating the next prototype.
9. Question the need to maintain models and documents that will not be referred to in the future.
10. Make sure you have management support to ensure stakeholder and resource availability during requirements definition.

It is important to recognize two realities: (1) it is impossible for stakeholders to describe an entire system before seeing the working software, and (2) it is difficult for stakeholders to describe quality requirements needed for the software before seeing it in action. Developers must recognize that requirements will be added and refined as the software increments are created. Capturing stakeholders' descriptions about what the system needs to do in their own words in a user story is a good place to begin.

If you can get stakeholders to define acceptance criteria for each user story, your team is off to a great start. It is likely that stakeholders will need to see a user story coded and running to know whether it has been implemented correctly or not. Therefore, requirements definition needs to be done iteratively and include the development of prototypes for stakeholder review.

Prototypes are tangible realizations of project plans that can be easily referenced by stakeholders when trying to describe desired changes. Stakeholders are motivated to discuss requirements changes in more concrete terms, which improves communication. It's important to recognize that prototypes allow developers to focus on short-term goals by only focusing on users' visible behaviors. It will be important to review

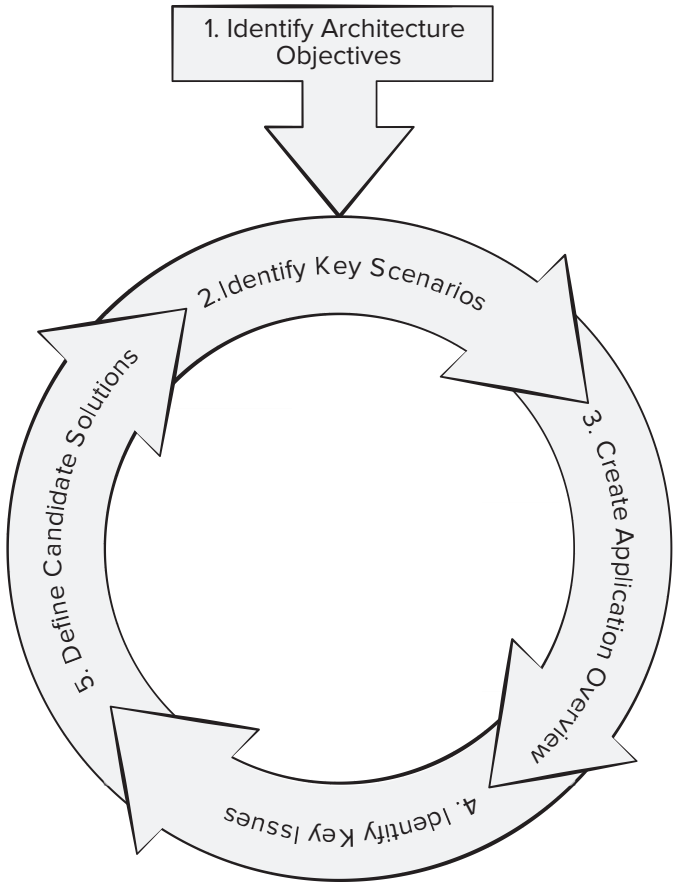
prototypes with an eye to the quality. Developers need to be aware that using prototypes may increase the volatility of the requirements if stakeholders are not focused on getting things right the first time. There is also the risk that creating prototypes before the software architectural requirements are well understood may result in prototypes that must be discarded, wasting time and resources [Kap15].

4.2 Preliminary Architectural Design

The decisions required to develop a solid architectural design are discussed in Chapter 10, but preliminary design decisions must often be made as requirements are defined. As shown in Figure 4.3, at some point in time, architectural decisions will need to be allocated to product increments. According to Bellomo and her colleagues [Bel14], early understanding of requirements and architecture choices is key to managing the development of large or complex software products.

FIGURE 4.3

Architectural design for prototype development



Requirements can be used to inform architecture design. Exploring the architecture as the prototype is developed facilitates the process of detailing the requirements. It is best to conduct these activities concurrently to achieve the right balance. There are four key elements to agile architectural design:

1. Focus on key quality attributes, and incorporate them into prototypes as they are constructed.
2. When planning prototypes, keep in mind that successful software products combine customer-visible features and the infrastructure to enable them.
3. Recognize that an agile architecture enables code maintainability and evolvability if sufficient attention is paid to architectural decisions and related quality issues.
4. Continuously managing and synchronizing dependencies between the functional and architectural requirements is needed to ensure the evolving architectural foundation will be ready just in time for future increments.

Software architecture decision making is critical to the success of a software system. The architecture of a software system determines its qualities and impacts the system throughout its life cycle. Dasanayake et al. [Das15] found that software architects are prone to making errors when their decisions are made under levels of uncertainty. Architects make fewer bad decisions if they can reduce this uncertainty through better architectural knowledge management. Despite the fact that agile approaches discourage heavy documentation, failing to record design decisions and their rationale early in the design process makes it hard to revisit them when creating future prototypes. Documenting the right things can assist with process improvement activities. Documenting your lessons learned is one of the reasons that retrospectives should be conducted after evaluating the delivered prototype and before beginning the next program increment. Reuse of previously successful solutions to architectural problems is also helpful and will be discussed in Chapter 14.

4.3 Resource Estimation

One of the more controversial aspects of using spiral or agile prototyping is estimating the time it will take to complete a project when it cannot be defined completely. It is important to understand before you begin whether you have a reasonable chance of delivering software products on time and with acceptable costs before you agree to take on the project. Early estimates run the risk of being incorrect because the project scope is not well defined and is likely to change once development starts. Estimates made when the project is almost finished do not provide any project management guidance. The trick is to estimate the software development time early based on what is known at the time and revise your estimates on a regular basis as requirements are added or after software increments are delivered. We discuss methods of estimating project scope in Chapter 25.

Let's examine how an experienced software project manager might estimate a project using the agile spiral model we have proposed. The estimates produced by this

method would need to be adjusted for the number of developers and the number of user stories that can be completed simultaneously.

1. Use historic data (Chapter 23), and work as a team to develop an estimate of how many days it will take to complete each of the user stories known at the start of the project.
2. Loosely organize the user stories into the sets that will make up each sprint[1] (Section 3.4) planned to complete a prototype.
3. Sum the number of days to complete each sprint to provide an estimate for the duration of the total project.
4. Revise the estimate as requirements are added to the project or prototypes are delivered and accepted by the stakeholders.

Keep in mind that doubling the number of developers almost never cuts the development time in half.

Rosa and Wallshein [Ros17] found that knowing initial software requirements at the start of a project provides an adequate but not always accurate estimate of project completion times. To get more accurate estimates, it is also important to know the type of project and the experience of the team. We will describe more detailed estimation techniques (e.g., function points or use case points) in Part Four of this book.

4.4 First Prototype Construction

In Section 2.5.2 we described the creation of prototypes as a means of helping the stakeholders move from statements of general objectives and user stories to the level of detail that developers will need to implement this functionality. Developers may use the first prototype to prove that their initial architectural design is a feasible approach to delivering the required functionality while satisfying the customer's performance constraints. To create an operational prototype suggests that requirements engineering, software design, and construction all proceed in parallel. This process is shown in Figure 4.1. This section describes steps that will be used to create the first prototypes. Details of best practices for software design and construction appear later in this book.

Your first task is to identify the features and functions that are most important to the stakeholders. These will help define the objectives for the first prototype. If the stakeholders and developers have created a prioritized list of user stories, it should be easy to confirm which are the most important.

Next, decide how much time will be allowed to create the first prototype. Some teams may choose a fixed time, such as a 4-week sprint, to deliver each prototype. In this case, the developers will look at their time and resource estimate and determine which of the high-priority user stories can be finished in 4 weeks. The team would then confirm with the stakeholders that the selected user stories are the best ones to include in the first prototype. An alternative approach would be to have the stakeholders

1 Sprint was described (Section 3.4) as a time period in which a subset of the system user stories will be delivered to the product owner.

and developers jointly choose a small number of high-priority user stories to include in the first prototype and use their time and resource estimates to develop the schedule to complete the first prototype.

The engineers working at National Instruments published a white paper that outlines their process for creation of a first functional prototype [Nat15]. These steps can be applied to a variety of software projects:

1. Transition from paper prototype to software design
2. Prototype a user interface
3. Create a virtual prototype
4. Add input and output to your prototype
5. Engineer your algorithms
6. Test your prototype
7. Prototype with deployment in mind

Referring to these seven steps, creating a *paper prototype* for a system is very inexpensive and can be done early in the development process. Customers and stakeholders are not usually experienced developers. Nontechnical users can often recognize what they like or do not like about a user interface very quickly once they see it sketched out. Communications between people are often filled with misunderstandings. People forget to tell each other what they really need to know or assume that everyone has the same understanding. Creating a paper prototype and reviewing it with the customer before doing any programming can help avoid wasted time building the wrong prototype. We will talk about several diagrams that can be used to model a system in Chapter 8.

Creating a *prototype user interface* as part of the first functional prototype is a wise idea. Many systems are implemented on the Web or as mobile applications and rely heavily on touch user interfaces. Computer games and virtual reality applications require a great deal of communication with end users to operate correctly. If customers find a software product easy to learn and use, they are more likely to use it.

Many misunderstandings between developers and stakeholders can be alleviated by beginning with a paper prototype of the user interface. Sometimes stakeholders need to see the basics of the user interface in action to be able to explain what they really like and dislike about it. It is less expensive to throw away an early user interface design than to finish the prototype and try to put a new user interface on top of it. Designing user interfaces that provide good user experiences is discussed in Chapter 12.

Adding input and output to your user interface prototype provides an easy way to begin testing the evolving prototype. Testing software component interfaces should be accomplished before testing the code that makes up the component's algorithms. To test the algorithms themselves, developers often use a "test frame" to ensure their implemented algorithms are working as intended. Creating a separate test frame and throwing it away is often a poor use of project resources. If properly designed, the user interface can serve as the test frame for component algorithms, thereby eliminating the effort required to build separate test frames.

Engineering your algorithms refers to the process of transforming your ideas and sketches into programming language code. You need to consider both the functional

requirements stated in the user story and the performance constraints (both explicit and implicit) when designing the necessary algorithms. This is the point where additional support functionality is likely to be identified and added to the project scope, if it does not already exist in a code library.

Testing your prototype demonstrates required functionality and identifies yet undiscovered defects before demonstrating it to the customer. Sometimes it is wise to involve the customer in the testing process before the prototype is finished to avoid developing the wrong functionality. The best time to create test cases is during requirements gathering or when use cases have been selected for implementation. Testing strategies and tactics are discussed in Chapters 19 through 21.

Prototyping with deployment in mind is very important because it helps you to avoid taking shortcuts that lead to creating software that will be hard to maintain in the future. This is not saying that every line of code will make it to the final software product. Like many creative tasks, developing a prototype is iterative. Drafts and revisions are to be expected.

As prototype development occurs, you should carefully consider the software architectural choices you make. It is relatively inexpensive to change a few lines of code if you catch errors before deployment. It is very expensive to change the architecture of a software application once it has been released to end users around the globe.

SafeHome



Room Designer
Considering First Prototype

The scene: Doug Miller's office.

The players: Doug Miller, software engineering manager; Jamie Lazar, software team member; Vinod Raman, software team member.

The conversation: (A knock on the door. Jamie and Vinod enter Doug's office.)

Jamie: Doug, you got a minute?

Doug: Sure, Jamie, what's up?

Jamie: We've been thinking about the scope of this *SafeHome* room design tool.

Doug: And?

Vinod: There's a lot of work that needs to be done on the back end of the project before people can start dropping alarm sensors and trying furniture layouts.

Jamie: We don't want to work on the back end for months and then have the project cancelled when the marketing folks decide they hate the product.

Doug: Have you tried to work out a paper prototype and review it with the marketing group?

Vinod: Well, no. We thought it was important to get a working computer prototype quickly and did not want to take the time to do one.

Doug: My experience is that people need to see something before they know whether they like it or not.

Jamie: Maybe we should step back and create a paper prototype of the user interface and get them to work with it and see if they like the concept at all.

Vinod: I suppose it wouldn't be too hard to program an executable user interface using the game engine we were considering using for the virtual reality version of the app.

Doug: Sounds like a plan. Try that approach, and see if you have the confidence you need to start evolving your prototype.